

	Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores Computação Distribuída (Época Normal) MEIC, MEIM	
Nº	Nome:	26/01/2022

EXAME ÉPOCA NORMAL (1ª PARTE, SEM CONSULTA, DURAÇÃO: 45 min., 11 valores)

Nas questões de 1 a 5 assinale as afirmações como verdadeira (V), falsa (F) ou não responde (—). Uma opção assinalada corretamente soma 0,25 valores e incorretamente desconta 0,125 valores (50% da cotação de cada alínea).

1. [1 val.] Sobre as características dos sistemas distribuídos

☐	As aplicações <i>stateless</i> facilitam o desenvolvimento de sistemas com elasticidade, dado que este tipo de aplicações não usam bases de dados
☐	O teorema CAP mostra que a exigência de consistência total, com tolerância a partições, diminui drasticamente a disponibilidade de um sistema
☐	Os balanceadores de carga são um exemplo de aplicação do conceito de transparência à replicação
☐	A adição de mais processadores/processos, resulta sempre num aumento linear do <i>speedup</i> , mesmo que haja uma parte da aplicação que não é paralelizável

2. [1 val.] Sobre a utilização de contratos, no desenvolvimento de aplicações distribuídas

☐	Em RMI e gRPC, os contratos dos serviços podem ser especificados numa linguagem de programação, independente da implementação do serviço
☐	O uso de contratos, facilita a interoperabilidade e desacoplamento entre serviços bastando unicamente conhecer as operações do contrato e a localização dos serviços
☐	Nos contratos gRPC é necessário especificar o tipo de exceções que cada operação pode retornar
☐	Um contrato <i>chatty</i> (muitas operações para cada funcionalidade) resulta em maior necessidade de comunicação e respetiva latência associada, bem como, maior probabilidade de falha

3. [1 val.] Relativamente ao desenvolvimento de aplicações em gRPC

☐	Numa operação com <i>stream</i> de cliente, o cliente tem a garantia que as mensagens chegam ao servidor pela ordem com que foram enviadas
☐	O método <i>onNext</i> de um <i>StreamObserver</i> tem vários parâmetros consoante o contrato <i>protobuf</i>
☐	Consoante o contrato <i>protobuf</i> , o desenvolvimento do servidor implica ter métodos diferentes para chamadas com <i>stub</i> bloqueante ou <i>stub</i> não bloqueante
☐	Numa operação com <i>stream</i> de cliente e <i>stream</i> de servidor, não é possível saber apenas pelo contrato qual a ordem das mensagens entre os dois <i>streams</i>

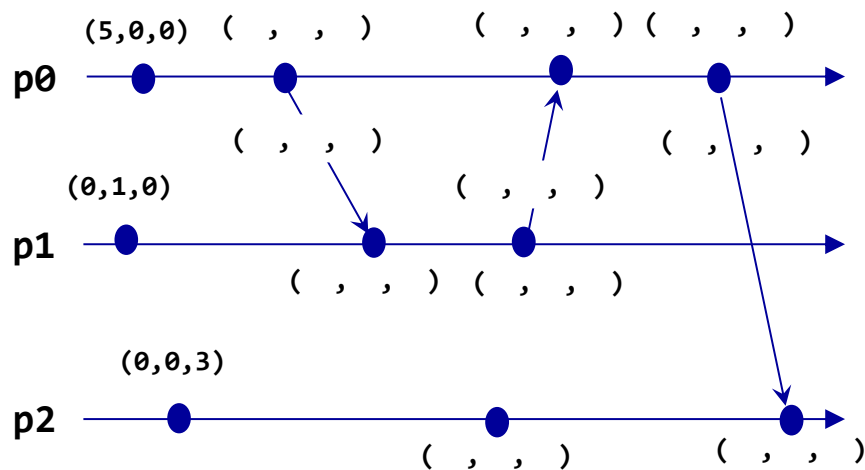
4. [1 val.] Sobre o sistema RabbitMQ

☐	Os <i>exchanges</i> não podem ser ligados (<i>bind</i>) uns aos outros para reencaminhar mensagens
☐	Um <i>exchange</i> pode estar associado (<i>bind</i>) a várias filas (<i>queues</i>)
☐	As filas (<i>queues</i>) podem entregar as mensagens a vários consumidores com uma política de <i>work-queue</i>
☐	Se um consumidor dá <i>acknowledge</i> a uma mensagem, o produtor da mesma recebe uma notificação

5. [1 val.] Sobre o algoritmo de consenso Raft

☐	O algoritmo assume que os participantes usam comunicação por grupos do tipo do <i>Spread Toolkit</i>
☐	O algoritmo de eleição é desencadeado quando um dos servidores (<i>followers</i>) não recebe o <i>heartbeat</i> do último líder conhecido
☐	Dado um cluster com 5 servidores, os clientes podem aceder a qualquer servidor, embora sejam redirecionados para o líder no caso de um pedido de atualização do <i>log</i> .
☐	O consenso é obtido pela eleição de um servidor líder, que tem a responsabilidade de coordenar a atualização das réplicas dos <i>logs</i>

6. [2 val.] Considere um sistema com 3 participantes (p_0 , p_1 , p_2) que utiliza na comunicação ponto a ponto relógios lógicos vetoriais. Assumindo que inicialmente os vetores dos relógios são os indicados na figura, $p_0=(5,0,0)$; $p_1=(0,1,0)$ e $p_2=(0,0,3)$, complete os valores dos relógios indicados por (, ,).



7. [2 val.] Considere um cenário cliente/servidor onde os clientes podem gerar, num período de tempo curto, milhares de pedidos, independentemente da existência de um ou vários servidores para processarem os pedidos e enviarem respostas aos clientes. Justifique as vantagens de usar o RabbitMQ na implementação desse cenário e indique um diagrama de arquitetura.

8. [2 val.] Considere uma imagem Docker com uma aplicação que necessita de um conjunto de dados de um ficheiro com nome *dados.dat*, existente numa diretoria do computador *host*. Descreva como é possível que múltiplos *containers* Docker, que executam a imagem, possam partilhar o ficheiro.